S&DS 365 / 665

Intermediate Machine Learning

Reinforcement Learning

November 2

Yale

Version of 2026/11/01 at 14:28

Goings on

- Assignment 3 due this Thursday, November 5
- Assignment 4 posted Wednesday, November 4
- Quiz 4 next Wednesday, November 11 (in class)
- Project proposals due next Thursday, November 12
  - Variational inference and VAEs
  - Undirected graphs and glasso
  - Graph Laplacians
- Wednesday: Deep Q-Learning

Outline for today

- Graph neural nets (quick recap)
- RL concepts
- Q-learning

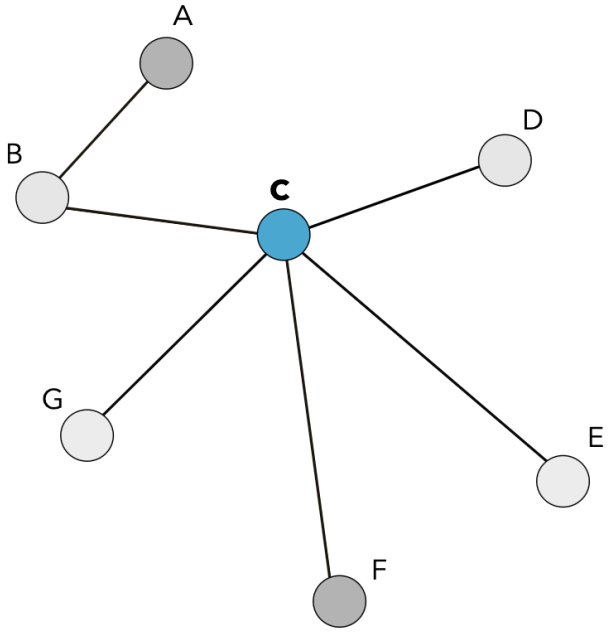
We'll go through background on RL quickly; please read up on RL if you have little prior familiarity

- Murphy, *Probabilistic machine learning*, Book 2, Chapter 35
- `https://probml.github.io/pml-book/book2.html`

Graph neural networks

Let’s quickly recap graph neural networks from last time

Graph Laplacian



Input Graph G

$$
\begin{array}{c} \\ A \\ B \\ \mathbf{C} \\ D \\ E \\ F \\ G \end{array}
\begin{array}{c}
\begin{array}{ccccccc} A & B & \mathbf{C} & D & E & F & G \end{array} \\
\begin{bmatrix}
1 & -1 & & & & & \\
-1 & 2 & -1 & & & & \\
 & -1 & 5 & -1 & -1 & -1 & -1 \\
 & & -1 & 1 & & & \\
 & & -1 & & 1 & & \\
 & & -1 & & & 1 & \\
 & & -1 & & & & 1
\end{bmatrix}
\end{array}
$$

Laplacian L of G

Filters and Laplacians

- A smoothing kernel relates each position to its neighbors
- An edge filter compares input positions in a particular direction
- Likewise, the graph Laplacian compares the value at each node to the values at neighboring nodes

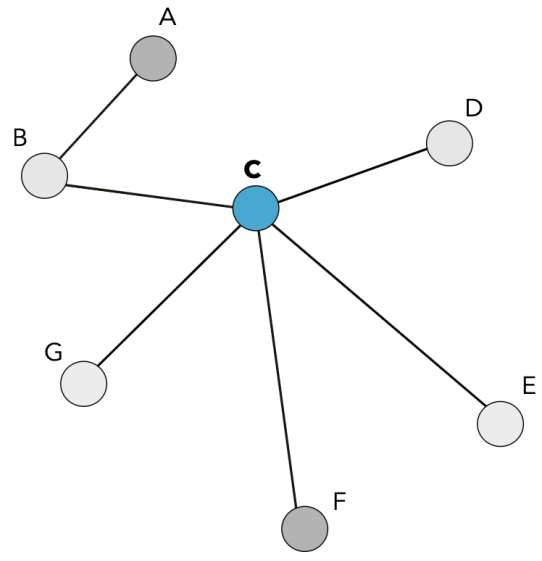
Normalized Laplacian

- Laplacian is $L = D - A$, where D is diagonal matrix of degrees, and A is the adjacency matrix
- Note: (i, j) entry of A^k is number of paths of length k from i to j
- Normalized Laplacian is

$$\Delta = I - D^{-1/2} A D^{-1/2}$$

- Normalized Laplacian is often more convenient to use

Graph Laplacian



Input Graph G

	A	B	**C**	D	E	F	G
A	1	−1					
B	−1	2	−1				
C		−1	5	−1	−1	−1	−1
D			−1	1			
E			−1		1		
F			−1			1	
G			−1				1

Laplacian L of G

$$\frac{1}{5}(Lx)_C = x_C - \frac{1}{5}(x_B + x_G + x_F + x_E + x_D)$$

$$= \text{value at } C \text{ minus average of neighbors}$$

Polynomials of the Laplacian

$$p_w(L) = w_0 I_n + w_1 L + w_2 L^2 + \cdots w_d L^d$$

If $\text{dist}(u, v) > i$ then the (u, v) entry of L^i is zero

- This is analogous to a CNN filter (smoothing kernel)
- The weights w_i play role of filter coefficients
- Degree d of polynomial plays role of the size of the kernel

Building layers

Let $h^{(k)}$ be the neurons at layer k.

We start with $h^{(0)} = x$, a value x_j at each node j

The next layer is

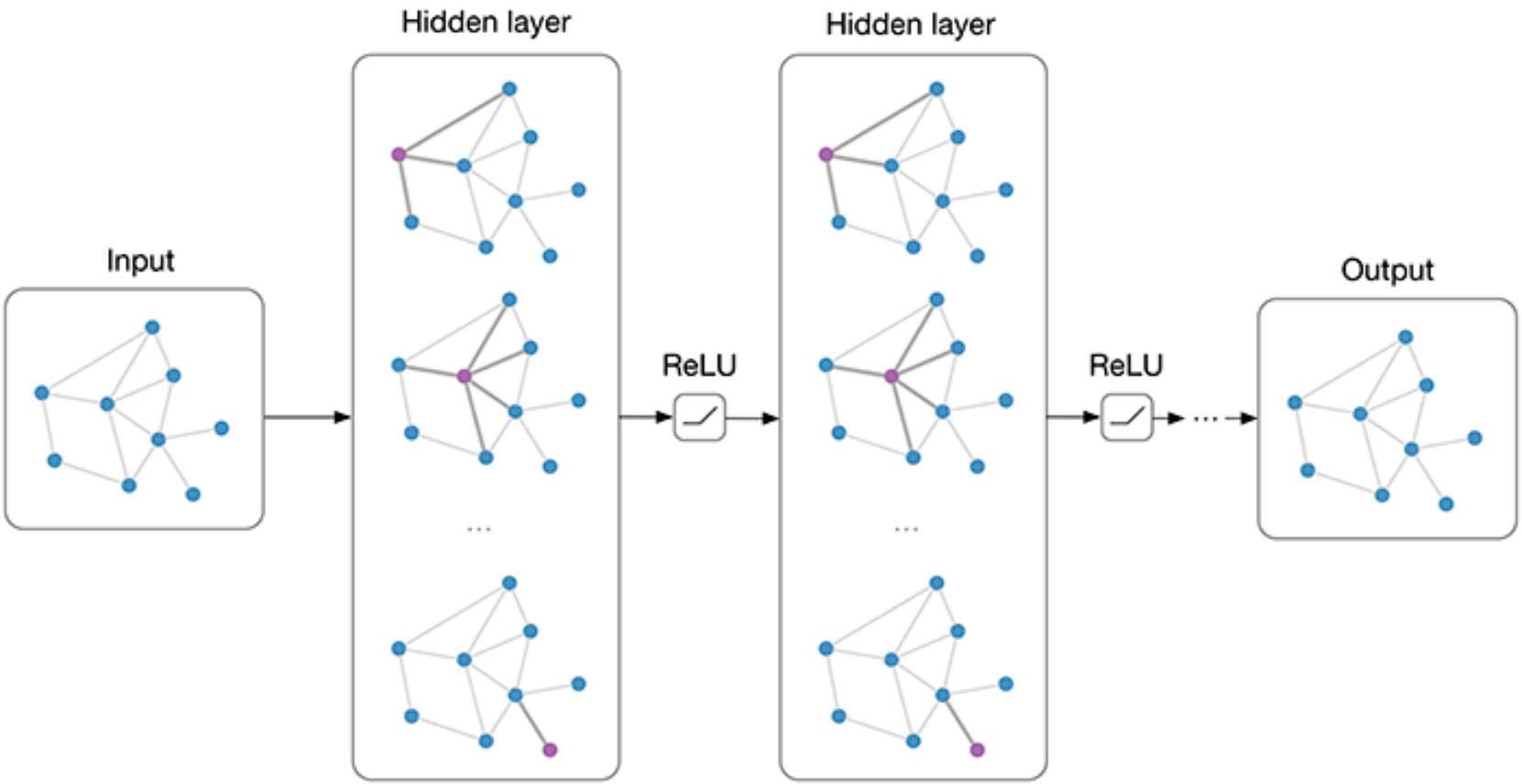
$$h^{(k+1)} = \varphi\left(p_{w^{(k)}}(L)h^{(k)}\right)$$

where φ is an activation function (applied component-wise)

---

https://distill.pub/2021/understanding-gnns/

Building layers



https://distill.pub/2021/understanding-gnns/. Another tutorial: https://tkipf.github.io/graph-convolutional-networks/

Building layers

- In this example, the first layer gives three “feature maps”

$$h_1 = \text{relu}(p_{w_1}(L)x)$$
$$h_2 = \text{relu}(p_{w_2}(L)x)$$
$$h_3 = \text{relu}(p_{w_3}(L)x)$$

- The final output is then computed as

$$y = \beta^T h = \beta_1 h_1 + \beta_2 h_2 + \beta_3 h_3$$

- The hidden states h_i and output y are vectors, with a component for each node in the graph.

Summary: Graph neural nets

- Certain data have natural graphical structure
- GNNs are analogues of CNNs for graphs
- Based on use of graph Laplacian
- Independent of ordering of nodes (equivariant)
- This is just a quick intro to a rich ML topic

Next: Reinforcement learning

Reinforcement learning

- An agent interacts with an environment
- The actions the agent takes change the state of the environment
- The agent receives rewards for each action, and seeks to maximize the total cumulative reward

Reinforcement learning is a framework for sequential decision making to achieve a long-term goal.

Reinforcement learning: Motivating examples

- Learning to walk or ride a bike
- A robot vacuum cleaning up the house
- Playing chess, backgammon, Atari games, etc.
- Drug discovery, personalized health, energy management
- Making an LLM useful for different tasks

Just two years ago...

RESEARCH

AI achieves silver-medal standard solving International Mathematical Olympiad problems

25 JULY 2024

AlphaProof and AlphaGeometry teams

Share

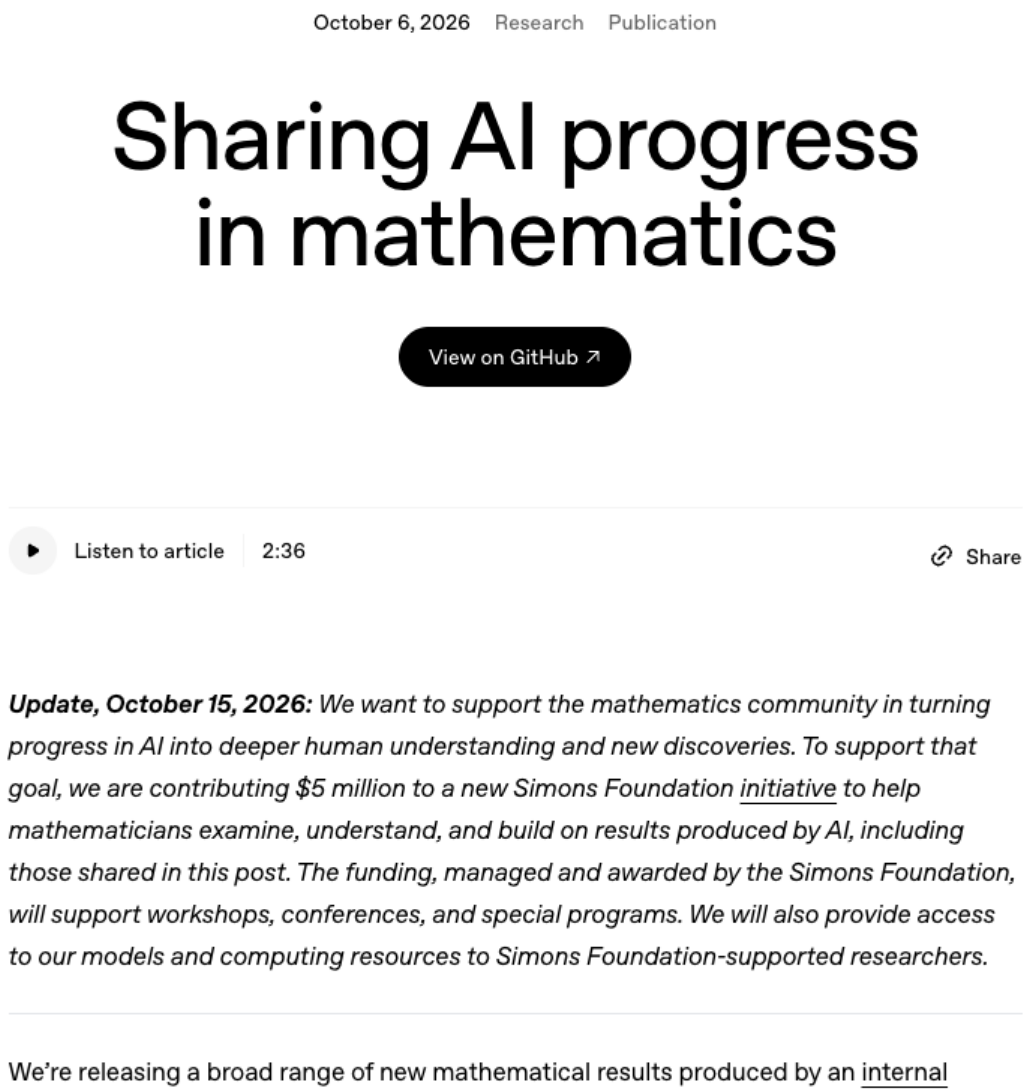
https://deepmind.google/discover/blog/ai-solves-imo-problems-at-silver-medal-level/


42
40
30
20
10
0
TOTAL
Gold
Silver
Bronze
Other
28 points
Our system
Points total
Human participant rank

AlphaProof: a formal approach to reasoning

AlphaProof is a system that trains itself to prove mathematical statements in the formal language Lean. It couples a pre-trained language model with the AlphaZero reinforcement learning algorithm, which previously taught itself how to master the games of chess, shogi and Go.

Just a month ago...



October 6, 2026 Research Publication

Sharing AI progress in mathematics

View on GitHub ↗

Listen to article 2:36 Share

Update, October 15, 2026: *We want to support the mathematics community in turning progress in AI into deeper human understanding and new discoveries. To support that goal, we are contributing $5 million to a new Simons Foundation initiative to help mathematicians examine, understand, and build on results produced by AI, including those shared in this post. The funding, managed and awarded by the Simons Foundation, will support workshops, conferences, and special programs. We will also provide access to our models and computing resources to Simons Foundation-supported researchers.*

We're releasing a broad range of new mathematical results produced by an internal frontier model.

http://openai.com/index/sharing-ai-progress-in-mathematics/

Reinforcement learning: Formalization

- The environment is in state s at a given time
- The agent takes action a
- The environment transitions to state $s' = \text{next}(s, a)$
- The agent receives reward $r = \text{reward}(s, a)$

Reinforcement learning: Formalization

- The environment is in state s at a given time
- The agent takes action a
- The environment transitions to state $s' = \text{next}(s, a)$
- The agent receives reward $r = \text{reward}(s, a)$

This is said to be a *Markov decision process*. It's "Markov" because the next state only depends on the current state and the action selected. It's a "decision process" because the agent is making choices of actions in a sequential manner.

Characteristics

- RL is inherently sequential
- In between supervised and unsupervised learning
- Agent can't act too greedily; needs to be strategic

The aim of RL is to learn to make optimal decisions from experience

Principles of RL

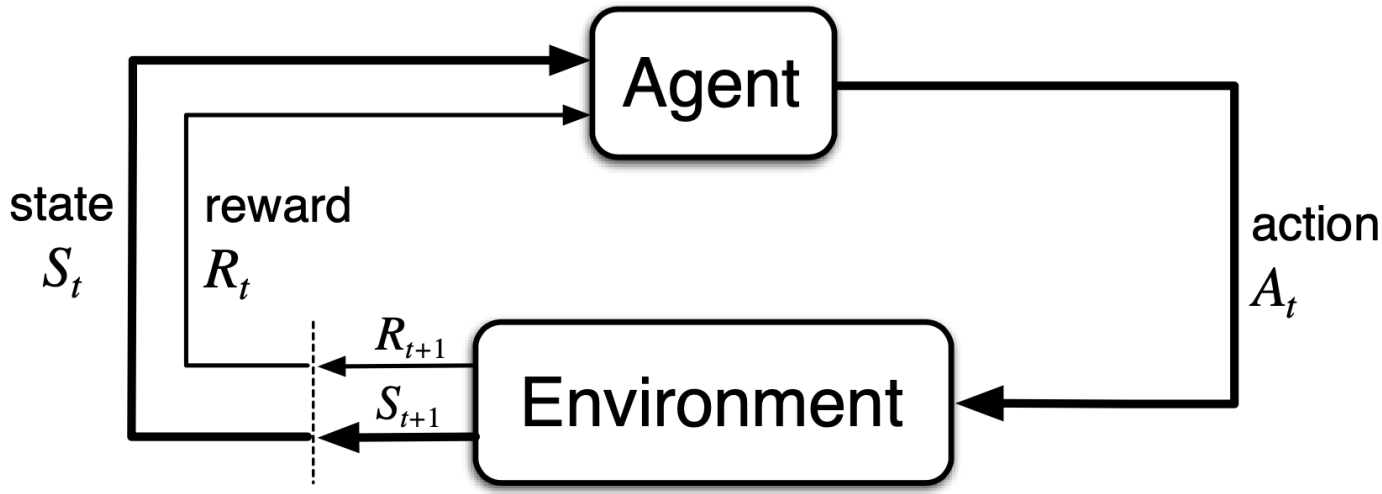
Some key RL concepts and principles:

Policy, reward signal, value function, model, Bellman equation

---

"Reinforcement Learning: An Introduction" (Second Edition), Richard S. Sutton and Andrew G. Barto

Principles of RL



Rewards and state transitions are probabilistic, in general

"Reinforcement Learning: An Introduction" (Second Edition), Richard S. Sutton and Andrew G. Barto

Principles of RL

Policy: A mapping from states to actions. An algorithm/rule to make decisions at each time step, designed to maximize the long term reward.

"Reinforcement Learning: An Introduction" (Second Edition), Richard S. Sutton and Andrew G. Barto

Principles of RL

Reward signal: The sequence of rewards received at each time step. An abstraction of “pleasure” (positive reward) and “pain” (negative reward) in animal behavior.

“Reinforcement Learning: An Introduction” (Second Edition), Richard S. Sutton and Andrew G. Barto

Principles of RL

Value function: A mapping from states to total reward. The total reward the agent can expect to accumulate in the future, starting from that state.

Rewards are short term. Values are expectations of future rewards.

"Reinforcement Learning: An Introduction" (Second Edition), Richard S. Sutton and Andrew G. Barto

Principles of RL

Model: Used for planning to mimic the behavior of the environment, to predict rewards and next states.

A *model-free* approach directly estimates a value function, without modeling the environment.

Analogous to distinction between generative and discriminative classification models

Taxi problem

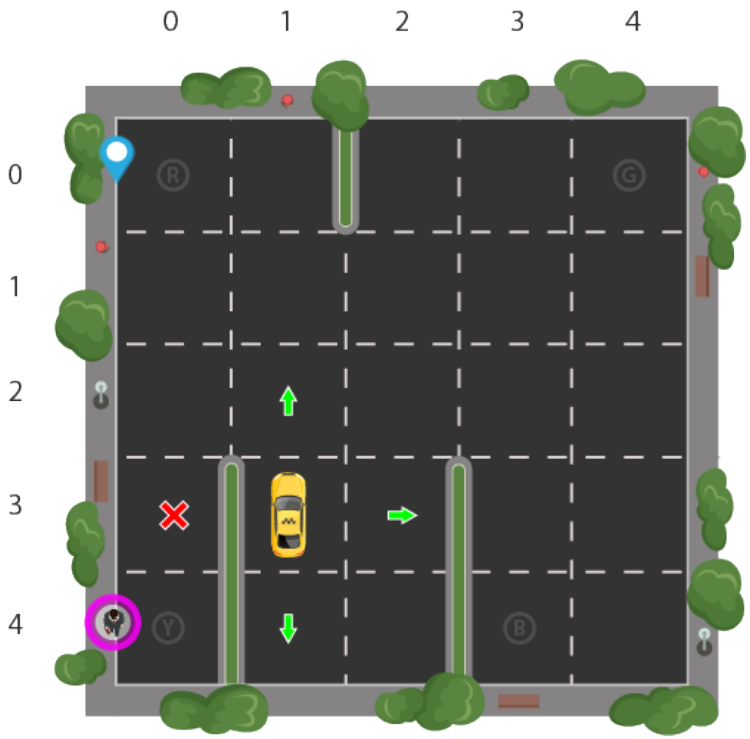
We'll introduce the important Q-learning algorithm with the toy "Taxi problem"

The code uses OpenAI gym

Our presentation follows the tutorial
https://www.learndatasci.com/tutorials/reinforcement-q-learning-scratch-python-openai-gym/

Taxi problem

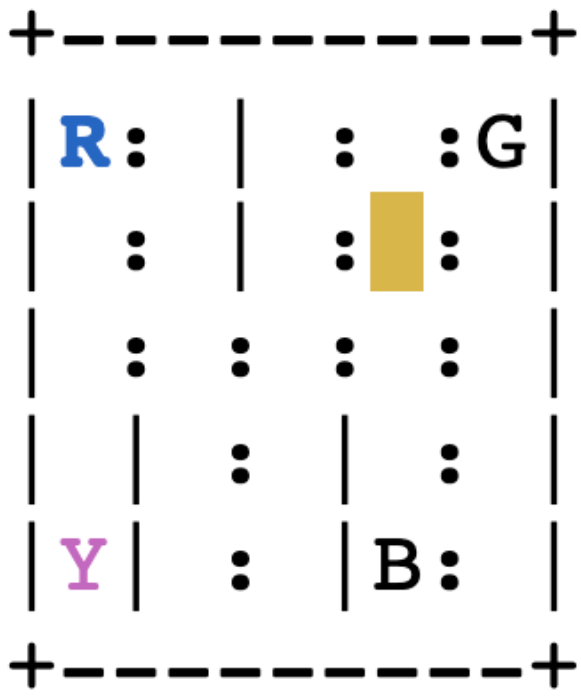
A taxicab drives around the environment, picking up and delivering a passenger at four locations



Taxi problem

A taxicab drives around the environment, picking up and delivering a passenger at four locations

”Ascii art” rendition:



Taxi problem: Description

- Four designated locations: R(ed), G(reen), Y(ellow), and B(lue)
- Taxi starts off at random square and passenger is at random location
- Taxi drives to passenger's location, picks up the passenger, drives to passenger's destination, drops off passenger
- Once the passenger is dropped off, the episode ends.

Taxi problem: State space

- 25 taxi positions
- 5 possible locations of passenger: At waiting location or in taxi
- 4 possible destination locations
- Total number of states: $25 \times 5 \times 4 = 500$

Taxi problem: State space

Passenger location coded as integers:

- 0: R(ed)
- 1: G(reen)
- 2: Y(ellow)
- 3: B(lue)
- 4: in taxi

Taxi problem: State space

Destinations coded as:

- 0: R(ed)
- 1: G(reen)
- 2: Y(ellow)
- 3: B(lue)

Taxi problem: State space

Agent actions coded as:

- 0: move south
- 1: move north
- 2: move east
- 3: move west
- 4: pickup passenger
- 5: drop off passenger

Taxi problem: State space

Rewards:

- Default reward per step: -1
- Reward for delivering passenger: +20
- Illegal “pickup” or “drop-off”: -10

Taxi problem: State space

State space represented as a tuple:

state = (taxi row, taxi column, passenger location, destination)

Q-learning

- Maintains a "quality" variable $Q(s, a)$ for taking action a in state s
- We'll sometimes refer to this as the *state-action value function*

Q-learning

- Maintains a “quality” variable $Q(s, a)$ for taking action a in state s
- We’ll sometimes refer to this as the *state-action value function*
- A measure of the cumulative rewards obtained by the algorithm when it takes action a in state s — the *value* of taking action a in state s

Q-learning

- Maintains a "quality" variable $Q(s, a)$ for taking action a in state s
- We'll sometimes refer to this as the *state-action value function*
- A measure of the cumulative rewards obtained by the algorithm when it takes action a in state s — the *value* of taking action a in state s
- Value should not be assessed purely based on the reward the action has in the current time step

Q-learning

- Maintains a "quality" variable $Q(s, a)$ for taking action a in state s
- We'll sometimes refer to this as the *state-action value function*
- A measure of the cumulative rewards obtained by the algorithm when it takes action a in state s — the *value* of taking action a in state s
- Value should not be assessed purely based on the reward the action has in the current time step
- Need to take into account the future rewards

Q-learning update

Update:

$$Q(s,a) \longleftarrow$$

$$Q(s,a) + \alpha\Big(\text{reward}(s,a) + \gamma \max_{a'} Q(\text{next}(s,a),a') - Q(s,a)\Big)$$

Q-learning update

Update:

$$Q(s,a) \longleftarrow$$
$$Q(s,a) + \alpha\Big(\text{reward}(s,a) + \gamma \max_{a'} Q(\text{next}(s,a),a') - Q(s,a)\Big)$$

- When action a is taken in state s, reward reward(s,a) is given
- Then, the algorithm moves to a new state next(s,a)

Q-learning update

Update:

$$Q(s,a) \longleftarrow$$
$$Q(s,a) + \alpha\Big(\text{reward}(s,a) + \gamma \max_{a'} Q(\text{next}(s,a),a') - Q(s,a)\Big)$$

- For example, if the taxi is location $(2,2)$ and takes the "West" action ($a = 3$), then there is a reward of -1, and the taxi moves to the new location (row, col) $= (2,1)$
- If cab is empty, it remains empty, and if it contains the passenger, the passenger remains.

Q-learning update

Update:

$$Q(s,a) \longleftarrow$$
$$Q(s,a) + \alpha\Big(\text{reward}(s,a) + \gamma \max_{a'} Q(\text{next}(s,a),a') - Q(s,a)\Big)$$

- Cumulative future reward of this action is $\max_{a'} Q(\text{next}(s,a),a')$
- Future rewards discounted by factor $\gamma < 1$
- Trades off short-term against long-term rewards
- A gradient *ascent* algorithm, with step size α

Let’s go to the notebook!

Bellman equation



The optimal value function is the largest expected discounted long term reward starting from that state.

---

https://en.wikipedia.org/wiki/Richard_E._Bellman

Bellman equation

Curse of dimensionality [edit]

Main article: Curse of dimensionality

The *curse of dimensionality* is an expression coined by Bellman to describe the problem caused by the exponential increase in volume associated with adding extra dimensions to a (mathematical) space. One implication of the curse of dimensionality is that some methods for numerical solution of the Bellman equation require vastly more computer time when there are more state variables in the value function. For example, 100 evenly spaced sample points suffice to sample a unit interval with no more than 0.01 distance between points; an equivalent sampling of a 10-dimensional unit hypercube with a lattice with a spacing of 0.01 between adjacent points would require 10^{20} sample points: thus, in some sense, the 10-dimensional hypercube can be said to be a factor of 10^{18} "larger" than the unit interval. (Adapted from an example by R. E. Bellman, see below.) [15]

---

https://en.wikipedia.org/wiki/Richard_E._Bellman

Bellman equation: Deterministic case

The optimality condition for the value function v_* is

$$v_*(s) = \max_a \Big\{ \text{reward}(s, a) + \gamma v_*(\text{next}(s, a)) \Big\}$$

Bellman equation: Deterministic case

The optimality condition for the Q-function is

$$Q_*(s,a) = \text{reward}(s,a) + \gamma \max_{a'} Q_*(\text{next}(s,a), a')$$

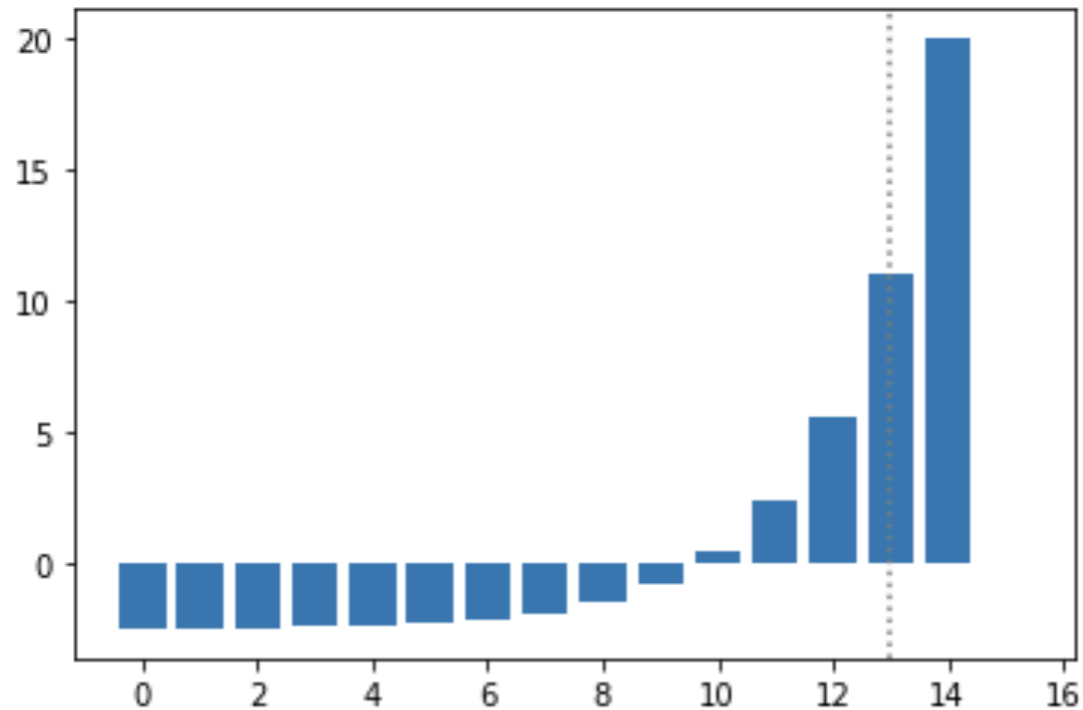
and then $v_*(s) = \max_{a'} Q_*(s,a')$

Q-learning update

Note how this makes sense in terms of the update rule:

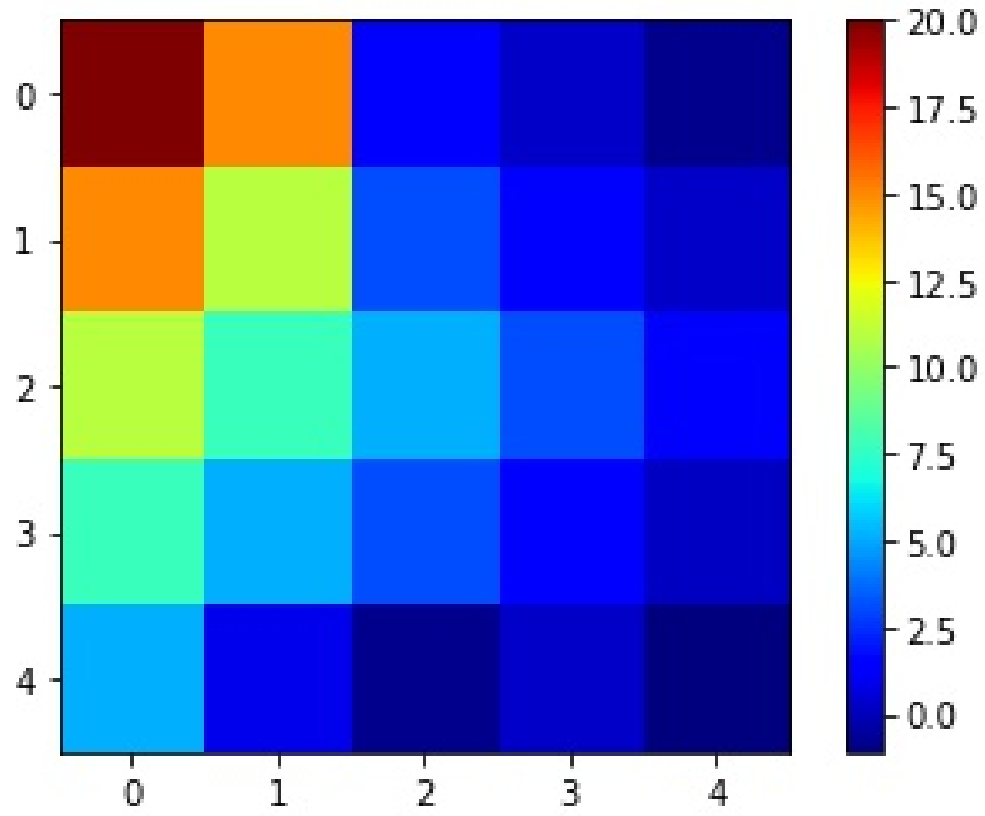
$$Q(s,a) \longleftarrow Q(s,a) + \alpha\Big(\text{reward}(s,a) + \gamma \max_{a'} Q(\text{next}(s,a), a') - Q(s,a)\Big)$$

```
+---------+
|R: | : :G|
| : | : : |
| : : : : |
| | : | : |
|Y| : |B: |
+---------+
  (East)
```



Question

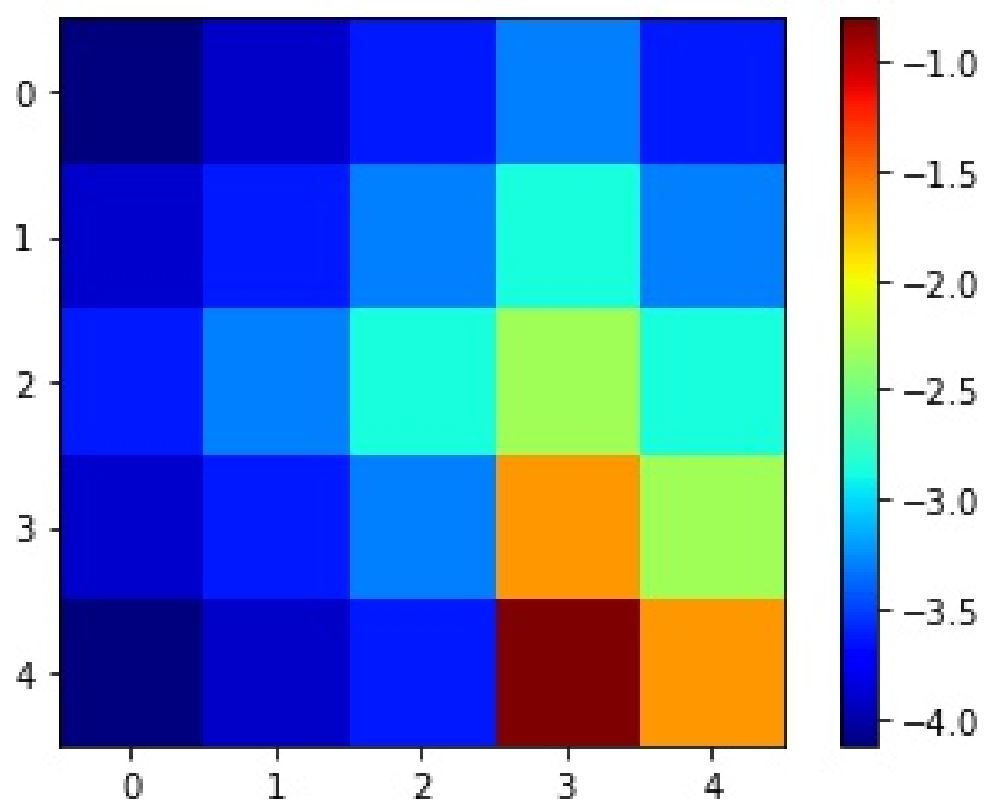
For a fixed passenger location and destination, the value function $v(\texttt{row}, \texttt{col})$ assigns a value to each of the $25 = 5 \times 5$ grid points.



Is the passenger waiting, or in the taxi?

Question

For a fixed passenger location and destination, the value function $v(\texttt{row}, \texttt{col})$ assigns a value to each of the $25 = 5 \times 5$ grid points.



Is the passenger waiting, or in the taxi?

Summary

- Reinforcement learning is a framework for sequential decision making to achieve a long-term goal
- The agent receives rewards for each action, and seeks to maximize the total cumulative reward
- The value of a state is the total reward the agent can expect to accumulate in the future, starting from that state
- Q-learning is an iterative algorithm that estimates the value of each state-action pair
- The Bellman equations are optimality conditions that characterize the value function